

Self-Referential Structures with Pointers (Self-Learning)

Q. Explain the concept of Structures in C. How can we use pointers with structures? Provide a code example to demonstrate its use.

*****Definition to Remember*****

1. Structures in C

* Groups multiple variables (called **members/fields**) of different types into one logical unit.

A **Stru**
Pointe
passing

* Ideal

2. Pointers with Structures

When a structure is passed to a function by value, a full copy is made - wasteful for large structures. Passing a pointer to the structure is more efficient.

* **Arrow Operator** (`->`) is used to access members via a pointer.

* `ptr->`

3. Self-Referential Structure

A structure that contains a pointer to another structure of the same type - the building block of Linked Lists and Trees.

struct N

4. Complete Code Example

```
#include <stdio.h>
```

```
#include
```

```
struct Student {
```

```
int roll_
```

```
int main() {
```

```
// Dyna
```

```
if (ptr == NULL) {
```

```
// Access members using Arrow Operator (->)
```

```
printf("Student Name: %s\n", ptr->name);
```

```
free(ptr); // Release allocated memory
```

5. Dot vs Arrow Operator

| Situation | Operator | Syntax |

*****Must-Write Points (for 10 marks)*****

*****Quick Recall*****

C Code: Self-Referential Structure Node Creation & Linked List Traversal

```
#include <stdio.h>
```

```
// Definition of Self-Referential Structure
```

```
// Function to traverse dynamic linked list
```

```
int main() {
```

```
// Dyna
```

```
// Assigning data & linking nodes
```

```
head->
```

```
second->data = 20;
```

```
second
```

```
third->data = 30;
```

```
third->n
```

```
printList(head);
```

```
// Free allocated memory
```

```
free(hea
```

```
return 0;
```

```
}
```